Toward Transparent and Incentive-Compatible Collaboration in Decentralized LLM Multi-Agent Systems: A Blockchain-Driven Approach

Minfeng Qi, *Member, IEEE*, Tianqing Zhu, *Member, IEEE*, Lefeng Zhang, Ningran Li, *Graduate Student Member, IEEE*, Yu-an Tan, *Senior Member, IEEE*, and Wanlei Zhou, *Life Fellow, IEEE*

***Abstract*—Large Language Models (LLMs) have enabled autonomous agents capable of complex reasoning and collaboration, yet coordinating such agents in decentralized environments remains challenging due to the lack of centralized control for aligning behavior and sustaining long-term cooperation. This paper addresses this challenge from a mechanism-design perspective and proposes a behavior-shaping incentive mechanism for decentralized LLM-based multi-agent systems. The mechanism explicitly models agent utility as a function of task rewards, capability mismatch, and workload, and couples short-term utility optimization with long-term trust formation through dynamic reputation updates and capability-weight adaptation. By jointly shaping task assignment probabilities, reputation evolution, and skill profiles over repeated interactions, the mechanism incentivizes cooperative behavior and discourages strategic misrepresentation or overload. To operationalize this incentive design in a trust-minimized setting, we implement a lightweight blockchain-based enforcement layer that provides verifiable identity binding, task assignment commitment, and immutable logging of incentive-relevant state transitions. We evaluate the proposed mechanism through 50-round simulations involving 20 agents and 100 tasks, using GPT-4-based agents integrated with Solidity smart contracts. The results demonstrate improved task success rates, stable utility distributions, and emergent agent specialization, highlighting the effectiveness of incentive-centric design for trustworthy coordination among decentralized LLM agents.**

***Index Terms*—Large language models, multi-agent systems, blockchain, smart contracts, and incentive mechanisms.**

I. Introduction

RECENT progress in Large Language Models (LLMs) has reshaped how we build intelligent systems. With strong abilities in reasoning, planning, and language understanding, LLMs have become the core of a new class of autonomous agents. These agents are now used in diverse areas such as scientific research, vulnerability detection, robotics, and complex problem solving [1], [2], [3], [4], [5], [6].

As tasks become more demanding, multi-agent systems (MAS) built on LLMs are emerging as a promising solution for scalable intelligence. By dividing work across multiple agents, MAS can support specialization and parallel execution that help tackle challenges beyond the reach of a single agent. Popular frameworks like MetaGPT [7], AutoGen [8], AgentVerse [9], AgentCoder [10] reflect this direction, enabling agents to plan, communicate, and coordinate. Yet, despite growing interest, a key question remains: How can we ensure that autonomous agents interact in a way that is *trustworthy*, *transparent*, and aligned with *incentives*, especially in decentralized settings?

Many existing LLM-MAS frameworks rely on strong assumptions. They often depend on centralized control, fixed communication rules, or trusted participants [11], [12], [13], [14]. These constraints make it difficult to apply such systems in open or adversarial environments, where agents may vary in design, pursue their own goals, or behave unpredictably. In particular, current frameworks face three major limitations:

- *Opaque collaboration mechanisms:* Many frameworks depend on off-chain coordination, where task assignment and communication happen outside any verifiable system [15]. This lack of transparency makes it hard to audit interactions or resolve disputes, especially in untrusted environments.
- *Missing incentive structures:* Without mechanisms for rewards or penalties, agents lack motivation to report their capabilities honestly or complete tasks with care [16]. This weakens collaboration, encourages poor task selection, and leads to unstable performance.
- *Limited scalability:* Fixed protocols and centralized controllers do not scale well as agent populations grow or environments change [17]. This creates bottlenecks in coordination and reduces the system's ability to operate effectively in real-world, open settings.

Several recent studies have attempted to address these shortcomings through different design strategies, yet each remains limited in scope. DeCoAgent [18] introduces a decentralized task marketplace, where smart contracts manage agent

Received 18 September 2025; revised 20 December 2025; accepted 22 January 2026. Date of publication 29 January 2026; date of current version 17 February 2026. This work was supported by the National Natural Science Foundation of China under Grant 62402008 and in part by the Macao Science and Technology Development Fund (FDCT) under Grant 0012/2025/RIC. Recommended for acceptance by Dr. Tong Wu. *(Corresponding author: Tianqing Zhu.)*

Minfeng Qi, Tianqing Zhu, Lefeng Zhang, and Wanlei Zhou are with the Faculty of Data Science, City University of Macau, Macau SAR 314100, China (e-mail: mfqi@cityu.edu.mo; tqzhu@cityu.edu.mo; lfzhang@cityu.edu.mo; wlzhou@cityu.edu.mo).

Ningran Li is with the School of Computer and Mathematical Sciences, The University of Adelaide, Adelaide, SA 5005, Australia (e-mail: ningran.li@adelaide.edu.au).

Yu-an Tan is with the Department of Computer Science and Engineering, Beijing Institute of Technology, Beijing 100811, China (e-mail: tan2008@bit.edu.cn).

Digital Object Identifier 10.1109/TNSE.2026.3659486

2327-4697 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

registration, capability discovery, and task publication. However, its design emphasizes standardization rather than dynamic incentives. BlockAgents [19] takes a different direction by embedding a Proof-of-Thought consensus protocol into the collaboration workflow. While this protocol improves robustness against Byzantine behaviors, it does not fully address heterogeneous participation. LLM-Net [20] proposes a blockchain-based expert network in which specialized LLM providers are selected according to immutable reputation records maintained by validators. Although this approach highlights specialization, it assumes that reputation can serve as a tamper-resistant proxy for agent quality. Taken together, these systems provide useful starting points but still fall short of simultaneously ensuring transparency, incentive alignment, and scalability in open multi-agent environments.

Our Approach: To address these challenges, we take a mechanism-design perspective and propose a *behavior-shaping incentive framework* for decentralized LLM-based multi-agent systems. Rather than treating blockchain as a mere logging substrate, our approach focuses on how agent utilities, task allocation, and long-term trust can be jointly designed to align agent behavior in the absence of centralized control. The key insight is that sustained cooperation among autonomous LLM agents emerges not from communication alone, but from carefully structured incentives that couple short-term rewards with long-term reputation and capability evolution.

At the core of our approach is an incentive mechanism that (i) assigns tasks probabilistically based on capability alignment, reputation, and workload, (ii) penalizes strategic overcommitment and capability mismatch through an explicit utility model, and (iii) continuously reshapes agent behavior via performance-driven reputation updates and capability adaptation over repeated interactions. This mechanism is designed to encourage truthful capability declaration, balanced participation, and specialization, while discouraging free-riding and opportunistic behavior.

To operationalize this incentive design in a trust-minimized setting, we implement a lightweight blockchain-based enforcement layer. Smart contracts are used selectively to bind agent identities, commit task assignments, and record incentive-relevant state transitions, while computation-intensive operations such as LLM inference and message exchange remain off-chain. This separation allows the system to maintain transparency and auditability without treating blockchain as a high-throughput coordination substrate.

To contextualize our design choices, Table I summarizes the key differences and trade-offs between existing decentralized LLM-MAS approaches and our incentive-centric framework.

Contributions: In summary, this work makes the following key contributions:

① We formulate decentralized LLM-based multi-agent coordination as a *behavior-shaping incentive design* problem and develop a concrete incentive mechanism that aligns agent behavior over repeated interactions through utility-based task allocation, performance-driven reputation updates, adaptive capability modeling, and workload-aware balancing, thereby encouraging cooperative behavior while discouraging strategic misrepresentation and overcommitment.

TABLE I
COMPARISON OF BLOCKCHAIN-ENABLED LLM-MAS

Dimension	DeCoAgent [18]	BlockAgents [19]	LLM-Net [20]	Ours
Task Decomposition	✗	✗	✗	✓
Assignment Logic	Marketplace (publish–claim)	Voting	Reputation	Utility Score
Agent Specialization	✗	✗	✓	✓
Incentive Mechanism	Fixed Reward	Stake-based (PoT)	Token Transfer	Behavior-Shaping
Behavior Tracking	▲	Reasoning Audit	▲	On-chain + Rep.
Utility Modeling	✗	✗	✗	✓
Performance Evolution	✗	✗	✗	✓
On-chain Transparency	✓	✓	✓	✓
Validation Type	Prototype (Ethereum)	Simulations (attack benchmarks)	Simulations	Prototype + simulation

Symbols: ✗ = Not supported, ▲ = Partially supported, ✓ = Fully supported.
Abbreviations: Rep. = Reputation; PoT = Proof-of-Thought.

TABLE II
COMPARISON OF TRADITIONAL MAS AND BLOCKCHAIN-ENHANCED MAS

Dimension	Traditional MAS	With Blockchain
Transparency	✗ Opaque	✓ Globally auditable
Immutability	✗ Logs rewritable	✓ Hash-anchored
Security	▲ Infra-dependent	✓ Cryptographic trust
Task Allocation	✗ typically centralized	✓ Fair and decentralized
Incentive	✗ Lacking mechanisms	✓ Tokenized rewards
Communication	✗ Trust-based	✓ Tamper-evident

② To operationalize the proposed incentive design in a trust-minimized setting, we implement a lightweight blockchain-based enforcement layer (Section IV) that provides verifiable identity binding, task assignment commitment, and immutable logging of incentive-relevant state transitions.

③ We conduct extensive simulation-based evaluations (Section V) with heterogeneous agents, demonstrating improved task success rates, stable utility distributions, and emergent specialization under decentralized conditions.

II. PRELIMINARIES

A. Blockchain Fundamentals

Blockchain is a distributed ledger technology that achieves data transparency, immutability, and high security through cryptographic techniques and decentralized consensus mechanisms [21], [22], [23]. These core characteristics make blockchain a valuable infrastructure component for enhancing transparency in traditional MAS. In our proposed framework, blockchain serves several main purposes: ensuring the transparency of task allocation, enforcing incentive protocol through smart contracts, and enabling trustworthy communication among agents (as summarized in Table II).

Blockchain data structure: The blockchain is organized as a linear, tamper-resistant sequence of blocks, where each block is linked to its predecessor via a cryptographic hash of the previous block's header [24], [25]. The hash of a block satisfies

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

TABLE III
EXPERIMENTAL ENVIRONMENT CONFIGURATION

Component	Specification
CPU	12-core AMD Ryzen 9 5900X @ 3.7GHz
RAM	32 GB DDR4
Operating System	Ubuntu 22.04 LTS
Blockchain Environment	Ganache CLI v7.8.0, Solidity v0.8.20, Hardhat v2.19
LLM API	OpenAI GPT-4 API (temperature = 0.2, max tokens = 1024)
Libraries	Web3.py, FastAPI, IPFS CLI, Numpy, Matplotlib

the following relationship:

$$H_{\text{block}} = \text{Hash}(H_{\text{prev}}, T, M_{\text{root}}, \text{Nonce}),$$

where H_{prev} denotes the hash of the previous block, enabling the chain-like linkage; T is a timestamp indicating when the block was created; M_{root} is the Merkle root summarizing all transactions within the block; and *Nonce* is a variable adjusted in proof-of-work (PoW) consensus to satisfy specific hash difficulty requirements. Any unauthorized alteration of block data will change H_{block}, thus invalidating the chain's integrity.

Blockchain uses a Merkle tree to organize transactions within each block [26]. A Merkle tree is a binary tree in which leaf nodes contain the hash of individual transactions, and non-leaf nodes contain hashes derived from their children:

$$M_{\text{node}} = \text{Hash}(M_{\text{left}}, M_{\text{right}}),$$

where M_{left} and M_{right} represent the hash values of the left and right child nodes, respectively. The Merkle root M_{root}, stored in the block header, enables secure verification of the inclusion of a specific transaction in the block. For instance, to verify whether a transaction is part of a block, one needs only to check if the computed hash path from the transaction up to the Merkle root matches the root recorded in the block header.

On-chain and off-chain storage: In practice, storing all data directly on-chain is inefficient and costly [27]. Therefore, blockchain-based systems often adopt a hybrid storage model that combines on-chain and off-chain storage. Off-chain data is typically managed using distributed file systems such as the InterPlanetary File System (IPFS) or conventional databases. The integrity of off-chain data is ensured by recording its cryptographic hash on-chain [28]. This enables verification via the following condition:

$$\text{Verify}(H_{\text{data}}, D_{\text{off-chain}}) = \begin{cases} \text{True}, & \text{if Hash}(D_{\text{off-chain}}) = H_{\text{data}}, \\ \text{False}, & \text{otherwise.} \end{cases}$$

This mechanism ensures that any tampering with off-chain data can be detected by a mismatch in its hash, thereby maintaining the verifiability and trustworthiness of externally stored information.

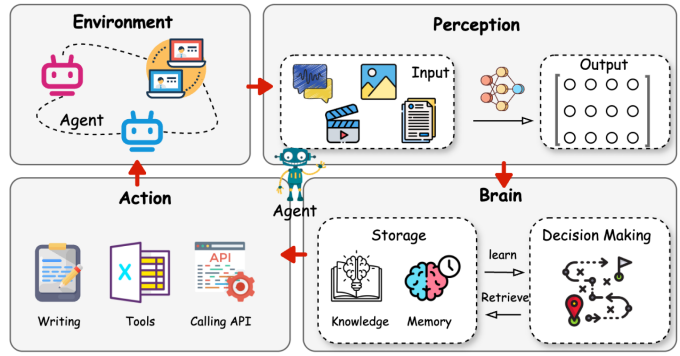


Fig. 1. LLM-MAS Closed-Loop Architecture. The agent continuously interacts with the environment through four major components: (i) *Perception Module* senses raw inputs from the environment; (ii) *Brain Module* manages reasoning, knowledge, and planning; (iii) *Action Module* executes decisions that affect the environment; and (iv) updated environmental states are fed back into perception, forming a closed loop.

B. Multi-Agent Systems

MAS are composed of multiple autonomous agents that interact within a shared environment to accomplish individual or collective objectives [29], [30]. These agents engage in a continuous cycle of perception, decision-making, and action, allowing them to adapt to dynamic contexts, cooperate or compete with other agents, and address complex problems beyond the capacity of a single agent [31], [32] (as illustrated in Fig. 1).

Perception: Perception refers to the agent's ability to sense and interpret environmental signals. This process transforms raw inputs (such as text queries, sensor data, or visual information) into structured internal representations that guide subsequent reasoning [33]. In the case of LLM agents, this often involves parsing natural language input, extracting relevant entities or instructions, and mapping them into task-relevant states or semantic structures.

Cognition and decision-making: At the core of each agent lies a reasoning module, often referred to as its cognitive engine or "brain." This component handles information processing, memory management, contextual reasoning, and task planning [34], [35]. It stores relevant knowledge, processes new inputs in light of that knowledge, and decides on appropriate actions. Reasoning can be implemented via predefined rules, symbolic logic, or data-driven models such as reinforcement learning. LLMs in particular provide strong generalization capabilities, allowing agents to respond flexibly to previously unseen instructions.

Action: After processing environmental input and selecting a course of action, the agent performs an action that either affects the external environment or influences other agents. This action may take various forms, including issuing API calls, generating responses, altering internal state, or sending structured messages [36]. The outcome of an action changes the environment, thereby initiating a new cycle of perception.

Environment: In MAS, the environment represents the external context with which agents continuously interact [37]. For LLM-based agents, this environment can be physical, such as a robotic workspace; digital, such as web APIs and databases; or interaction-specific, such as a user's intent or a conversational

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

history [38]. The environment serves as both the source of stimuli for agent perception and the target of agent actions.

Agent loop: This perception-decision-action process forms a continuous loop that governs the agent's behavior. As the environment evolves, agents re-perceive, re-evaluate, and re-act, enabling adaptation to dynamic conditions and coordination with other agents [39]. This loop is particularly important in collaborative settings where agent behaviors must remain synchronized with environmental changes and peer activity.

III. Behavior-Shaping Incentive Mechanism

As a central innovation of our framework, we design a behavior-shaping mechanism (Alogrithm 1) that couples short-term utility optimization with long-term trust shaping. Specifically, agent behavior is dynamically regulated through a protocol that (i) computes utilities based on reward, workload, and capability mismatch, (ii) updates reputation scores via performance-sensitive smoothing, and (iii) adjusts capability weights according to task-specific outcomes. By integrating these components, the mechanism not only rewards high-quality task execution but also penalizes poor performance, thereby steering agents toward cooperative behavior over interactions.

Mechanism-design perspective: We emphasize that this module is not merely an implementation component of the system, but a mechanism design for decentralized agent collaboration in the economic and game-theoretic sense. In our setting, each agent is a self-interested decision-making entity that strategically chooses which tasks to pursue and how much effort to exert under limited capacity. The mechanism shapes these strategic choices through an explicit utility structure and repeated-interaction feedback: it aligns individual incentives with system-level objectives by coupling immediate payoffs with future selection advantages induced by reputation and capability updates. The blockchain layer serves as an enforcement substrate that makes the mechanism verifiable and tamper-evident (e.g., immutable logs and auditable updates).

A. Utility-Based Incentive Function

In our framework, each participating entity is modeled as an autonomous agent operating in a decentralized environment. Let $\mathcal{A} = \{A_1, A_2, \ldots, A_n\}$ denote the set of all agents. Each agent $A_i \in \mathcal{A}$ is characterized at time step t by a profile consisting of three core components: its reputation score $\rho_i^t \in [0,1]$, a capability vector $\mathbf{w}_i^t = (w_{i,1}, \ldots, w_{i,d}) \in \mathbb{R}_{\geq 0}^d$, and a workload level $\ell_i^t \in \mathbb{N}$. The reputation score reflects the agent's historical performance quality, the capability vector encodes proficiency across d skill domains, and the workload level records the number of active or pending tasks assigned to the agent.

Tasks are periodically issued into the environment. Each task T_j is defined by a reward value $R_j > 0$, a required capability vector $\mathbf{r}_j \in \{0,1\}^d$, and an expected execution window or deadline τ_j. The binary capability vector $\mathbf{r}_j$ specifies which skill dimensions are required for completing the task, i.e., $r_{j,k} = 1$ indicates that skill k is necessary.

Algorithm 1: Behavior-Shaping Incentive Mechanism for Decentralized Agents Coordination.

Input: Task set $\mathcal{T}$, Agent set $\mathcal{A}$, capability taxonomy $\mathcal{T}_{\text{cap}}$
Output: Updated ρ_i^{t+1} and $\boldsymbol{w}_i^{t+1}$ for all $A_i \in \mathcal{A}$
foreach *task* $T_j \in \mathcal{T}$ **do**
  Retrieve reward R_j, required tag vector $\mathbf{r}_j$, and deadline τ_j;
  foreach *agent* $A_i \in \mathcal{A}$ **do**
    Compute binarized capability $\hat{\mathbf{w}}_i^t$ from $\boldsymbol{w}_i^t$ using threshold θ;
    Compute cost: $c_i^t(T_j) = \beta \cdot \ell_i^t + \gamma \cdot \|\mathbf{r}_j - \hat{\mathbf{w}}_i^t\|_1$;
    Compute utility: $U_i^t(T_j) = \pi_i^t(T_j) \cdot R_j - c_i^t(T_j)$;
    Compute assignment probability $\pi_i^t(T_j)$ using scoring function based on ρ_i^t, $\boldsymbol{w}_i^t$, and ℓ_i^t;
  Assign task T_j to agent A_{i^*} with highest $\pi_{i^*}^t(T_j)$ and log assignment;
  Execute T_j and collect performance metrics: $q_{i^*}^t$, $d_{i^*}^t$, and tag-wise $s_{i^*,k}^t$;
  Compute task score: $S_{i^*}^t = \alpha \cdot q_{i^*}^t + \delta \cdot (1 - d_{i^*}^t)$;
  Update reputation: $\rho_{i^*}^{t+1} = \lambda \cdot \rho_{i^*}^t + (1-\lambda) \cdot S_{i^*}^t$;
  foreach *tag* $\mathsf{Tag}_k \in \mathcal{T}_j$ **do**
    Update capability weight: $w_{i^*,k}^{t+1} = \mu \cdot w_{i^*,k}^t + (1-\mu) \cdot s_{i^*,k}^t$;
  Set $\ell_{i^*}^{t+1} \leftarrow \ell_{i^*}^t - 1$;

To allocate tasks effectively, the system defines a task assignment policy $\pi_i^t(T_j) \in [0,1]$ (9), which represents the probability of assigning task T_j to agent A_i at time t. This assignment probability is derived from the scoring function (8) that considers the agent's reputation, capability match, and current workload.

The expected utility that agent A_i receives at time t when being evaluated for task T_j is defined as:

$$U_i^t(T_j) = \pi_i^t(T_j) \cdot R_j - c_i^t(T_j), \tag{1}$$

where $\pi_i^t(T_j) \cdot R_j$ denotes the expected reward the agent may earn, and $c_i^t(T_j)$ denotes the cost of executing the task. This cost function encapsulates both the agent's current workload and the degree of mismatch between its skills and the task requirements. Formally, we define:

$$c_i^t(T_j) = \beta \cdot \ell_i^t + \gamma \cdot \left\|\mathbf{r}_j - \hat{\mathbf{w}}_i^t\right\|_1, \tag{2}$$

where $\beta > 0$ and $\gamma > 0$ are system-defined coefficients, ℓ_i^t is the current workload, and $\hat{\mathbf{w}}_i^t \in \{0,1\}^d$ is the binarized capability vector derived from $\mathbf{w}_i^t$ by thresholding, i.e., $\hat{w}_{i,k}^t = 1$ if $w_{i,k}^t \geq \theta$, and 0 otherwise. The L_1 norm $\|\mathbf{r}_j - \hat{\mathbf{w}}_i^t\|_1$ quantifies the number of required skills that the agent lacks.

Thus, based on (2) and (1), the agent's utility function becomes:

$$U_i^t(T_j) = \pi_i^t(T_j) \cdot R_j - \beta \cdot \ell_i^t - \gamma \cdot \left\|\mathbf{r}_j - \hat{\mathbf{w}}_i^t\right\|_1. \tag{3}$$

This formulation implies that an agent's incentive to accept a task increases with a higher assignment probability $\pi_i^t(T_j)$, larger reward R_j, and better capability alignment with the task

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

(i.e., a smaller skill mismatch). In contrast, agents are discouraged from accepting tasks when they are overloaded (ℓ_i^t is large) or underqualified (i.e., many components of $\mathbf{r}_j$ are not satisfied by $\hat{\mathbf{w}}_i^t$).

The incentive structure is thus designed to promote truthful reporting of capabilities and discourage strategic misrepresentation. Since utility is explicitly penalized by capability mismatch and task overload, agents have no rational incentive to overstate their abilities or accept tasks beyond their capacity. Moreover, as will be demonstrated in subsequent sections, this utility function integrates naturally with our dynamic reputation update rule and capability-weight learning mechanism, forming a self-regulating incentive structure that rewards cooperative and capable behavior over repeated interactions.

B. Reputation Update Rule

To incentivize high-quality behavior and maintain long-term agent accountability, our framework integrates a dynamic reputation update mechanism. Each agent $A_i \in \mathcal{A}$ maintains a scalar reputation score $\rho_i^t \in [0,1]$, which is updated over time based on its observed performance across assigned tasks. This reputation score plays a central role in influencing future task assignment probabilities and serves as a trust indicator in agent selection procedures.

Let $S_i^t \in [0,1]$ denote the performance score that agent A_i receives upon completing a task at time step t. This score is computed based on the task outcome, and can be determined either through automated metrics (e.g., accuracy, latency) or feedback from the task issuer. To ensure stability and gradual adaptation, we define the reputation update rule using an exponential moving average (EMA) as follows:

$$\rho_i^{t+1} = \lambda \cdot \rho_i^t + (1-\lambda) \cdot S_i^t, \tag{4}$$

where $\lambda \in [0,1)$ is a smoothing coefficient that determines how strongly past reputation influences future values. A higher value of λ yields more inertia (slower change), while a lower λ allows faster responsiveness to recent performance.

This update rule ensures that an agent's reputation gradually converges toward its average task performance. Since $S_i^t \in [0,1]$ and $\rho_i^t \in [0,1]$, the update equation preserves boundedness: $\rho_i^{t+1} \in [0,1]$ for all t. The EMA design naturally implements a forgetting mechanism, allowing agents to recover from past errors over time while still penalizing consistently poor behavior.

The performance score S_i^t is computed as a weighted aggregation of multiple task-specific indicators:

$$S_i^t = \alpha \cdot q_i^t + \delta \cdot (1 - d_i^t), \tag{5}$$

where $q_i^t \in [0,1]$ is the task quality score (e.g., correctness or user rating), $d_i^t \in [0,1]$ is a normalized delay ratio (actual time divided by deadline), and $\alpha + \delta = 1$ are weighting coefficients. This formulation penalizes delayed or substandard executions and rewards timely, high-quality completions.

A key property of the reputation rule is that it aligns with the agent's utility function U_i^t (3). Since task allocation probabilities π_i^t are influenced by ρ_i^t, maximizing long-term utility requires agents to improve their reputation. Let the expected cumulative utility over a horizon T be denoted by:

$$\mathbb{E}[\mathcal{U}_i] = \sum_{t=1}^{T} \mathbb{E}[U_i^t(T_j)]. \tag{6}$$

Substituting the utility expression from the previous section and noting that $\pi_i^t \propto \rho_i^t$, it becomes clear that maximizing $\mathbb{E}[\mathcal{U}_i]$ requires maintaining a high ρ_i^t across t. This creates a natural incentive for rational agents to pursue good behavior over time.

C. Capability Weight Adjustment

In addition to reputation, each agent $A_i \in \mathcal{A}$ maintains a dynamic capability vector $\boldsymbol{w}_i^t \in [0,1]^K$, where each element $w_{i,k}^t$ quantifies the agent's expertise or reliability in a specific skill dimension $k \in \{1,2,\ldots,K\}$. These capability weights serve as fine-grained indicators that complement the scalar reputation score and facilitate task-specific matching in heterogeneous multi-agent environments.

Let Tag_k denote the k-th capability tag in the global capability taxonomy $\mathcal{T} = \{\mathsf{Tag}_1, \mathsf{Tag}_2, \ldots, \mathsf{Tag}_K\}$. The value $w_{i,k}^t \in [0,1]$ indicates the proficiency level of agent A_i on tag Tag_k at time t. Initially, each agent declares a self-assessed capability vector $\boldsymbol{w}_i^0$ during registration. However, the system dynamically updates these values based on the agent's historical task performance in each capability domain.

For each completed task T_j with required tag set $\mathcal{T}_j \subseteq \mathcal{T}$, the agent receives a capability-specific performance score $s_{i,k}^t \in [0,1]$ for each tag $\mathsf{Tag}_k \in \mathcal{T}_j$. This local score reflects how well the agent performed the sub-tasks associated with that capability dimension. The weight update rule is again modeled using exponential smoothing:

$$w_{i,k}^{t+1} = \mu \cdot w_{i,k}^t + (1-\mu) \cdot s_{i,k}^t, \quad \text{for all } \mathsf{Tag}_k \in \mathcal{T}_j, \tag{7}$$

where $\mu \in [0,1)$ is the capability parameter that governs the sensitivity to new observations. Tags that are not involved in the task ($\mathsf{Tag}_k \notin \mathcal{T}_j$) retain their previous weights: $w_{i,k}^{t+1} = w_{i,k}^t$.

The use of tag-specific smoothing allows agents to improve (or degrade) their perceived skill levels over time in a way that is tightly aligned with actual task performance. Agents who consistently perform well in certain domains will see corresponding capability weights increase, making them more likely to be selected for similar tasks in the future. Conversely, agents with poor or unstable performance in a domain will experience weight decay, reducing their assignment likelihood for those tags.

D. On-Chain Enforcement Via Smart Contracts

To ensure verifiability and autonomy of coordination among agents, our framework leverages on-chain enforcement through smart contracts. All state transitions, including agent registration, task assignment, behavioral logging, performance evaluation, and reputation updates, are conducted on-chain to benefit from blockchain's immutability and auditability.

Each smart contract maintains the relevant state variables and enforces lifecycle transitions via publicly verifiable transactions.

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

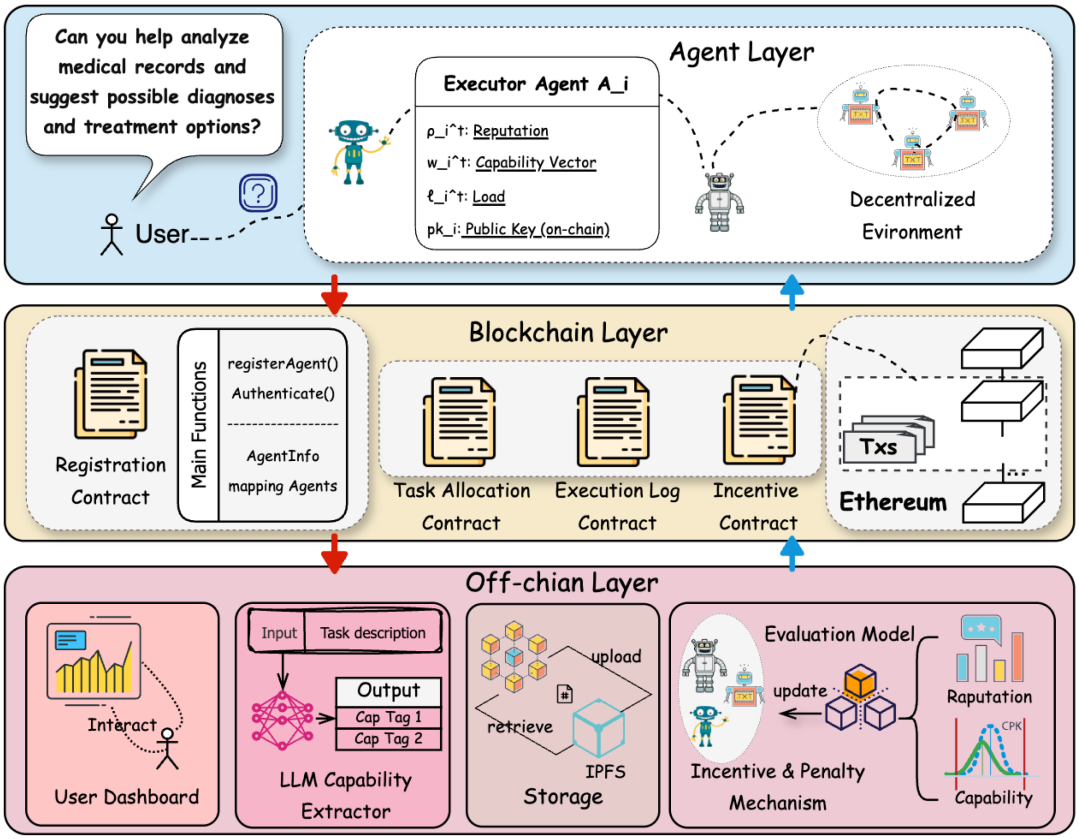


Fig. 2. System architecture of blockchain-based LLM multi-agent collaboration. The system is structured into three logical layers. The *Off-chain Layer* handles user interactions, LLM capability extraction, storage, and performance evaluation. The *Blockchain Layer* supports agent registration, task allocation, execution logging, and incentive mechanisms via smart contracts deployed on Ethereum. The *Agent Layer* consists of autonomous agents operating in a decentralized environment, each maintaining reputation, capability, load, and public key information.

When an agent A_i joins the system, its declared capabilities $\mathcal{C}_i$, public key, and initial reputation r_i^0 are stored through the invocation of the registerAgent function. All agent-specific operations thereafter, such as authentication, workload updates, and reputation adjustments, are tracked and enforced through designated contract interfaces.

Tasks T_j are registered on-chain via submitTask, which stores the task's metadata, including capability requirements $\mathsf{requiredTags}_j$ and execution constraints. The assignment policy $\pi_i^t(T_j)$ is computed on-chain by evaluating agent scores based on capability match, current load, and historical reputation. The selected agent A_i is recorded through a state update and confirmed by emitting a TaskAssigned event.

During task execution, all behavioral evidence, including task acceptance, intermediate updates, and result submission, is reported using the logAction interface. Each log entry binds a specific action to an agent-task pair, forming a verifiable tuple $\langle A_i, T_j, \mathsf{action}, t\rangle$. These logs serve as audit trails.

Upon task completion, performance evaluation and reputation updates are enforced on-chain via the updateReputation function. The function takes the task score $\mathsf{TS}_{i,j}$ as input and computes the updated reputation r_i^{t+1} using a predefined rule. This ensures that all reputation transitions are transparent and irrevocable.

IV. System Design

To enable transparent and autonomous coordination among LLM-based agents, we implement the proposed behavior-shaping incentive mechanism (Section III) through a smart contract-driven interaction protocol (cf. Fig. 2). In this section, we briefly describe the supporting system components that make the mechanism executable in a decentralized setting.

The protocol contains three functional modules: (1) agent registration and authentication, (2) communication and messaging, (3) decentralized task allocation.

A. Registration and Authentication Module

This module binds each agent to a verifiable cryptographic identity and restricts participation to registered agents, providing the basis for accountable reputation and capability tracking.

Registration: Each agent generates a local public–private key pair and registers by calling registerAgent, submitting *AgentID*, *PublicKey* (using secp256k1), *Role*, *Capabilities* (tags), and an optional initial *Reputation*. The contract validates uniqueness and input format, stores metadata in a mapping indexed by the agent address, initializes the workload counter, and emits an AgentRegistered event.

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

```
{
  "message_id": "UUID",
  "sender": "Agent_A",
  "receiver": "Agent_B",
  "timestamp": "2025-06-12 10:30:00",
  "message_type": "TaskAssignment",
  "task_id": "Task_UUID",
  "capability_tags": ["Tag1", "Tag2"],
  "status": "InProgress",
  "message_body": {
    "description": "Task or status update
        description",
    "data": "Associated data or context"
  },
  "signature": "digital_signature"
}
```

Listing 1: Standard Message Structure.

Authentication and eligibility checks: To participate in interactions, an agent proves ownership of its key via signature-based challenge by invoking authenticate; the contract verifies the signature via ecrecover against the registered address. For task-related operations, the contract enforces a lightweight capability gate via verifyCapabilities, checking whether the agent's registered tags satisfy the task's required tags.

B. Communication and Messaging Module

Communication provides the evidence and coordination signals needed for decentralized collaboration. Messages are signed by senders to ensure authenticity, and timestamps help mitigate replay attempts and support basic ordering.

Hybrid logging: To balance auditability with cost, we adopt hybrid logging: message hashes and essential metadata are anchored on-chain, while full payloads can be stored off-chain (e.g., IPFS) when needed. This design preserves verifiability without incurring excessive on-chain storage overhead.

Message schema: Messages follow a standardized schema that includes *sender*, *receiver*, *timestamp*, *message_type*, *message_body*, and *signature* (Listing 1). The unified schema enables automatic parsing and downstream validation (e.g., task eligibility checks and capability matching).

C. Task Allocation Module

Task allocation selects agents based on capability match, reputation, and current workload, thereby instantiating the assignment policy used by the incentive mechanism. Let $\pi_i^t(T_j) \in [0,1]$ denote the probability that agent A_i is assigned to task T_j at time t. We compute a utility-driven score:

$$S_{i,j}^t = \lambda_1 \cdot \mathsf{CapMatch}(\mathcal{C}_j^{\text{req}}, \boldsymbol{w}_i^t) + \lambda_2 \cdot \rho_i^t - \lambda_3 \cdot \ell_i^t, \tag{8}$$

where $\mathsf{CapMatch}(\mathcal{C}_j^{\text{req}}, \boldsymbol{w}_i^t) = \frac{1}{|\mathcal{C}_j^{\text{req}}|} \sum_{k \in \mathcal{C}_j^{\text{req}}} w_{i,k}^t$ is the average tag-wise capability weight over required capabilities. The assignment probability is computed by a softmax over eligible agents:

$$\pi_i^t(T_j) = \frac{\exp(S_{i,j}^t)}{\sum_{A_k \in \mathcal{A}_j} \exp(S_{k,j}^t)}, \tag{9}$$

ensuring that higher-scoring agents are more likely to be selected while still allowing exploration.

Task representation and updates: Tasks are described in natural language and mapped to required capability tags $\mathcal{C}_j^{\text{req}}$ by an LLM-based capability extractor. After assignment, the selected agent's workload ℓ_i^t is updated and the task state is recorded for auditability. Upon completion, task outcomes trigger reputation and capability updates as specified in Section III, closing the feedback loop.

Optional decomposition: For tasks exceeding a complexity threshold θ_c, tasks can be decomposed into subtasks with inherited capability requirements, and the same allocation rule is applied per subtask. In our implementation, task complexity is measured using a lightweight structural metric derived from the task description, defined as a weighted combination of (i) the number of required capability tags, (ii) the estimated number of atomic action steps inferred from the instruction, and (iii) the presence of explicit sequential dependencies (e.g., conjunctions such as "and then"). Formally, a task is marked as complex if $\mathsf{Complexity}(T_j) > \theta_c$, where $\mathsf{Complexity}(\cdot)$ is computed off-chain during task parsing.

V. Experimental Setup

A. System Implementation

Our system consists of three main components: (1) the LLM-based agent layer, (2) the smart contract coordination layer, and (3) the user-facing frontend.

(1) Agent layer: Each agent is instantiated as an independent process equipped with access to OpenAI's GPT-4 API. Agents communicate via signed messages (using ECDSA over secp256k1) and locally maintain their reputation scores, capability profiles, and decision-making modules.

(2) Smart contract layer: The blockchain backend is implemented using the Ethereum Virtual Machine (EVM). All contract logic, including agent registration, task assignment, and reputation updates, is encoded in Solidity. We used Hardhat for development and deployed contracts on a local Ganache testnet to simulate realistic blockchain execution.

(3) Frontend and middleware: We implemented a lightweight React-based frontend that allows users to submit tasks, monitor agent performance, and visualize blockchain logs. A FastAPI-based backend mediates between agent behavior and smart contract calls, including transaction signing, gas estimation, and off-chain capability evaluation.

The entire system was developed and tested on a machine with the following specifications (Table III):

B. Simulated Task Environment

Task corpus and capability taxonomy: We derive our task set from a modified subset of the ALFRED benchmark [40], a well-known environment used in multi-agent learning and embodied task planning. ALFRED provides hierarchical natural language instructions grounded in realistic household environments, which are particularly suitable for simulating multi-skill task execution in distributed agent systems. For our purposes,

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
CAPABILITY TAG TAXONOMY USED FOR AGENT PROFILING

Tag	Description
Tag_1	Object recognition and classification
Tag_2	Spatial reasoning and planning
Tag_3	Language understanding (instruction parsing)
Tag_4	Grasping and manipulation
Tag_5	Path planning and navigation
Tag_6	Scene understanding (layout/context extraction)
Tag_7	Task decomposition and sequencing
Tag_8	Temporal reasoning (event ordering, not deadlines)
Tag_9	Knowledge grounding (external inference)
Tag_{10}	Environment interaction via API calls or actuators

TABLE V
TASK ALLOCATION METRICS (AVG OVER 50 ROUNDS)

Metric	Mean	Std	Min	Max	R1	R25	R50
Success Rate (%)	86.77	4.10	78.35	95.21	80.15	88.40	94.49
Failure Rate (%)	13.23	4.10	4.79	21.65	19.85	11.60	5.51
Mean Quality	0.82	0.042	0.72	0.91	0.75	0.81	0.89
Quality StdDev	0.042	0.013	0.025	0.068	0.061	0.042	0.030
Agent Utility	5.02	0.81	3.52	6.43	3.52	5.01	6.43
Cap Match Score	0.84	0.03	0.76	0.90	0.78	0.83	0.89
Alloc Delay (steps)	1.42	0.31	1.00	2.10	1.75	1.41	1.08
Retry Count	0.86	0.58	0	2	2	1	0
Agent Load	2.51	0.45	1.73	3.62	2.10	2.43	2.88
Load StdDev	0.96	0.22	0.55	1.30	1.22	0.89	0.76

we extract 100 representative tasks involving both atomic goals (e.g., "pick up the book") and composite objectives (e.g., "put the red mug on the kitchen table and turn off the lights").

Each task is automatically annotated with 2–4 relevant capability tags drawn from a curated global taxonomy $\mathcal{T} = \{\mathsf{Tag}_1, \mathsf{Tag}_2, \ldots, \mathsf{Tag}_{10}\}$. This taxonomy captures diverse functional dimensions and serves as the semantic interface between task descriptions and agent skill profiles (as presented in Table IV).

Each task T_j is thus encoded as a binary capability requirement vector $\mathbf{r}_j \in \{0,1\}^{10}$, where each component $r_{j,k} = 1$ indicates that the task demands skill Tag_k. For example, a composite instruction such as "pick up the knife and place it on the stove" may require Tag_1 (object recognition), Tag_4 (manipulation), and Tag_5 (navigation).

Agent initialization and skill specialization: We simulate a population of 20 autonomous agents $\mathcal{A} = \{A_1, A_2, \ldots, A_{20}\}$, each configured with distinct capability vectors, reputation scores, and initial workload states. These agents emulate heterogeneous expertise profiles common in decentralized multi-agent systems.

To construct the initial capability distribution, we assume that agents exhibit specialization rather than uniform proficiency. For each agent A_i, we sample its continuous capability vector $\mathbf{w}_i^0 \in [0,1]^{10}$ from a Beta distribution $\mathsf{Beta}(\alpha = 2, \beta = 5)$. This asymmetric distribution is skewed toward lower values, reflecting the intuition that most agents are only proficient in a few domains, while few are highly versatile. The choice of parameters $\alpha = 2, \beta = 5$ yields moderate sparsity and aligns with empirical patterns observed in expert population models.

To enhance inter-agent comparability, all vectors are min-max normalized across each agent's 10-dimensional skill profile. We then binarize each vector using a global threshold $\theta = 0.4$, resulting in a discrete skill declaration vector $\hat{\mathbf{w}}_i^0 \in \{0,1\}^{10}$, where $\hat{w}_{i,k}^0 = 1$ if and only if $w_{i,k}^0 \geq \theta$. This binarized representation serves as the basis for capability matching during task allocation and closely mirrors real-world agent registration processes, where skill declarations are coarse-grained rather than continuous.

Each agent is initialized with a neutral reputation score $\rho_i^0 = 0.5$, reflecting a lack of prior task performance history. This value serves as the starting point for our exponential reputation update process and ensures fairness in early-round allocations. To simulate asynchronous participation and limited concurrency, we assign initial workload levels $\ell_i^0 \in \{0,1,2\}$ sampled uniformly at random.

Additionally, we store each agent's public key and declared capability profile on-chain via the registerAgent contract function during system bootstrap. This guarantees that all subsequent identity verification, capability claims, and behavior logs are cryptographically verifiable and traceable within the immutable system ledger.

Simulation execution: The simulation spans 50 discrete time steps (referred to as rounds), each modeling a cycle of task issuance, agent bidding, execution, and state updates.

At the beginning of each round $t \in \{1, \ldots, 50\}$, the environment injects a new batch of 5 tasks $\{T_1^t, \ldots, T_5^t\}$ into the task queue. These tasks are selected from the ALFRED-derived benchmark pool and retain their annotated capability vectors $\mathbf{r}_j$ and reward values R_j. The tasks are broadcast to all agents via the emitTaskBatch smart contract function and stored on-chain.

Each agent $A_i \in \mathcal{A}$ evaluates its expected utility $U_i^t(T_j)$ for each task T_j in the current batch. This evaluation incorporates three elements: (i) the agent's current workload ℓ_i^t, (ii) capability alignment (as measured by $\|\mathbf{r}_j - \hat{\mathbf{w}}_i^t\|_1$), and (iii) the task assignment policy $\pi_i^t(T_j)$, as defined in (9). The resulting utility values are thresholded, and agents only bid for tasks where $U_i^t(T_j) > 0$, ensuring rational bidding behavior.

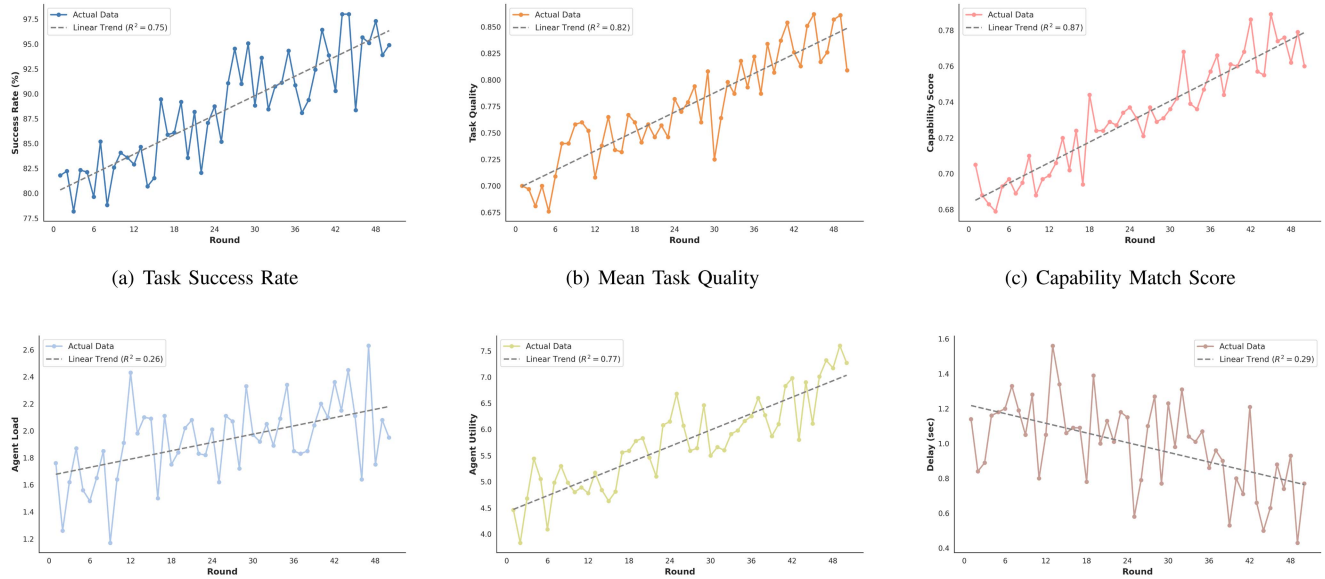
Agents submit bids through the submitBid function, attaching a signed message that includes: the agent's identifier, the task ID, the binarized capability vector $\hat{\mathbf{w}}_i^t$, the computed utility, and a digital signature using the agent's private key. The smart contract receives all bids and executes the assignment procedure on-chain by invoking the scoring-based task assignment policy.

Once assignments are finalized, each selected agent proceeds to "execute" its task in a simulated performance environment. Task outcome is probabilistically determined by a ground-truth simulator, where the success probability $p_{\text{success}} \in [0,1]$ depends on:

$$p_{\text{success}}(A_i, T_j) = \eta \cdot \text{CapMatch}(\hat{\mathbf{w}}_i^t, \mathbf{r}_j) \cdot (1 - \zeta \cdot \ell_i^t), \quad (10)$$

where $\eta \in (0,1]$ is a base success rate coefficient, $\zeta \in [0,1]$ penalizes overload, and CapMatch computes the normalized overlap between required and capabilities. The result is a binary task success flag (1 = success, 0 = failure), which drives

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.



(a) Task Success Rate (b) Mean Task Quality (c) Capability Match Score

(d) Agent Utility (e) Agent Load (f) Allocation Delay

Fig. 3. Performance trends across 50 simulation rounds for multiple task allocation metrics in LLM-based multi-agent systems. Each subplot visualizes actual measurements and a fitted linear trend line with its R^2 value to show consistency and correlation.

subsequent updates to the agent's reputation ρ_i^t and capability weights $w_{i,k}^t$, based on the update rules detailed in Section III.

All major system actions, including emitTaskBatch, submitBid, logAction, and updateReputation, are implemented as Ethereum, compatible Solidity smart contracts and executed over a local private Ethereum network (based on Hardhat + Ganache). Each action emits a corresponding blockchain event with encoded parameters, enabling full auditability via standard tools like Web3 log parsers.

VI. Results and Analysis

A. Task Allocation Efficiency

To assess the performance of our proposed task allocation mechanism, we conducted a simulation of 50 rounds of interactions between 100 tasks and 20 agents. The metrics reported in Table V reflect task completion success, quality, agent utility, and load balancing behavior.

Task success rate (cf. Fig. 3(a)): Across 50 simulation rounds, the average task success rate reached **86.77%**, with an initial value of 80.15% in early rounds that improved to 94.49% in later rounds. This increasing trend illustrates the positive feedback loop built into the system: agents with higher reputations and more refined capabilities were prioritized in task selection, resulting in more successful executions over time.

Mean task quality (cf. Fig. 3(b)): The mean task quality score $\mathsf{TS}_{j,i}$, derived from both correctness and delay metrics, steadily improved from an initial average of 0.75 to a final value of 0.89, with an overall mean of **0.82**. This shows that tasks were increasingly handled by agents with better skill alignment. We observed a decline in task quality standard deviation from 0.061 to 0.030 across the simulation horizon, indicating reduced variability in performance. This trend demonstrates that the system successfully discouraged suboptimal task-agent matches, as poorly performing or overloaded agents were deprioritized in the assignment process.

Capability match score (cf. Fig. 3(c)): The capability match score, which quantifies how well assigned agents align with the required skills of each task, exhibited steady improvement over the 50-round simulation. In early rounds, the average capability match score hovered around 0.68–0.70, indicating moderate alignment with some degree of skill mismatch. However, as agents refined their bidding strategies and specialized in high-reward domains, this score improved noticeably. By round 25, the average capability match rose to approximately 0.735, and by round 50, it reached 0.765, with a maximum round value of **0.794**. This upward trend suggests that agents increasingly learned to bid selectively for tasks that matched their strongest skill domains.

Agent utility (cf. Fig. 3(d)): The average agent utility per round reached **5.02**, increasing from 3.52 in the early simulation to 6.43 in the final rounds. This gain reflects the reinforcement of incentive compatibility: agents that reported capabilities truthfully and maintained high reputations were rewarded with higher-value task allocations and better-matched workloads.

Load distribution (cf. Fig. 3(e)): Throughout the simulation, we monitored the system's ability to balance load across the agent population. In the early rounds, a few agents dominated task execution due to favorable initial capabilities. However, as the reputation system evolved, task allocation became more evenly distributed across eligible agents, avoiding persistent overload on specific individuals. The final rounds showed smoother workload curves across agents.

Task allocation delay (cf. Fig. 3(f)): The average delay between task issuance and final assignment was approximately 0.99 rounds, with variation between 0.43 and 1.56. Initial delay in Round 1 (1.14) reflects the system's cold start and lack of optimized matching. Over time, delays decreased, with mid-

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

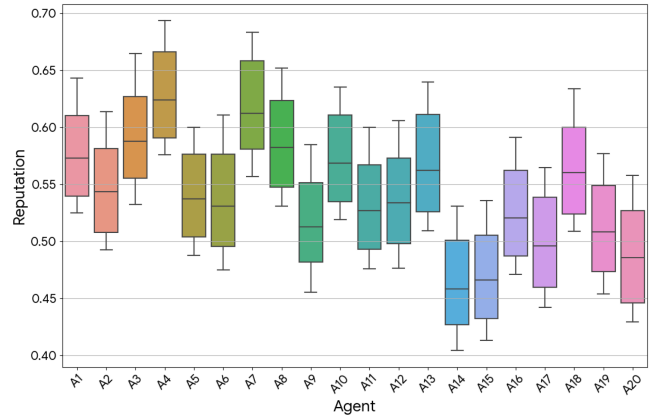
and late-stage rounds (e.g., Round 25 and 50) showing improved responsiveness (0.58 and 0.77 respectively). This confirms that our on-chain task assignment policy reduces latency as the system evolves.

B. Reputation and Capability Evolution

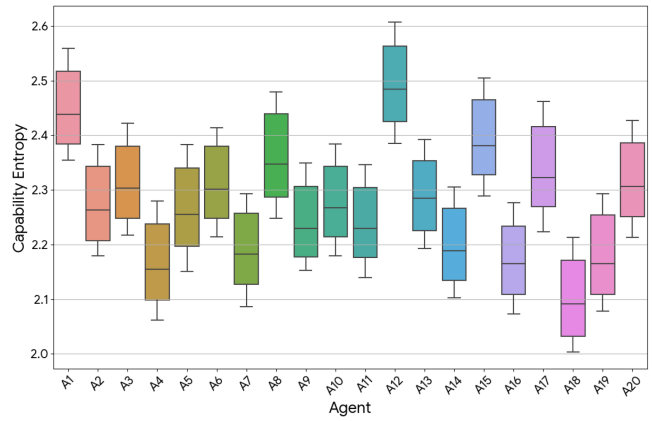
To evaluate the impact of our incentive and learning mechanisms on agent behavior, we analyze the evolution of reputation scores ρ_i^t and capability entropy over 50 simulation round. These metrics respectively reflect agent reliability and skill specialization dynamics.

Reputation dynamics: Contrary to sharp bifurcation or polarization, agent reputation scores demonstrated relatively stable and bounded evolution. By round 50, the mean final reputation across the 20 agents was 0.488 (Std $= 0.044$), with the highest and lowest values being 0.558 and 0.429, respectively. No agents crossed the high-performance threshold (> 0.8), nor did any fall below the critical underperformance boundary (< 0.3). This convergence suggests a moderate equilibrium state under our reward-compatible mechanism, potentially resulting from balanced task distribution and relatively homogenous agent behavior. Fig. 4(a) visualizes these dynamics, showing bounded but individualized reputation trajectories.

Capability specialization: Capability vectors exhibited similarly stable characteristics. The entropy of each agent's skill weight vector $\boldsymbol{w}_i^t$ was monitored to assess specialization tendencies. At initialization, agents presented a mean entropy of 2.31 bits. After 50 rounds, the mean entropy remained consistent at 2.315 bits (Std $= 0.073$), with no agent dropping below the specialization threshold of 1.4 bits. These findings suggest that agents retained generalist profiles, with limited capability concentration. This behavior may be attributed to relatively uniform feedback across capabilities, indicating room for introducing stronger task-tag selectivity or differential reinforcement. Fig. 4(b) illustrates these entropy patterns.



(a) Reputation Per Agent



(b) Capability Entropy Per Agent

Fig. 4. Per-agent analysis of reputation and capability diversity. (a) shows the reputation variance, while (b) presents entropy as a proxy for multi-skill potential.

C. Agent Incentivization and Specialization

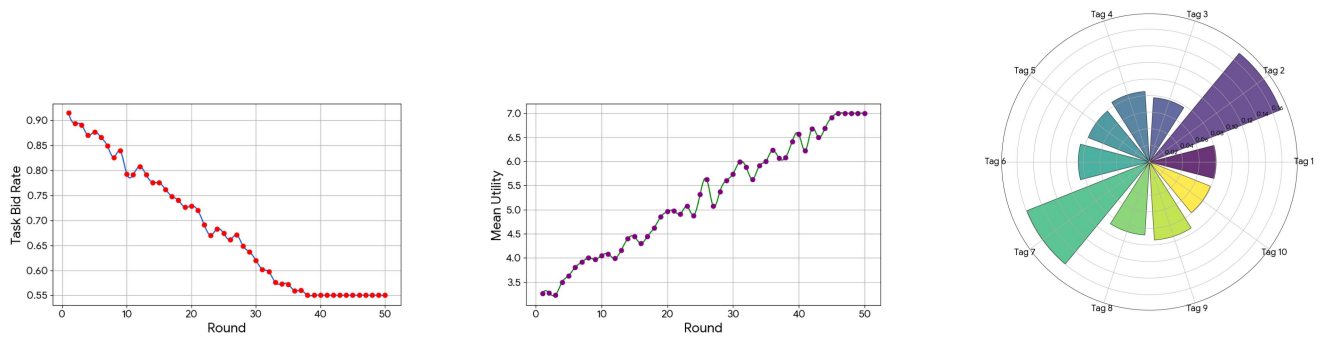
To assess whether our incentive model drives rational agent behavior and task-domain specialization, we tracked key behavioral metrics over 50 simulation rounds, including task bidding behavior, utility trends, and tag-specific dominance distributions.

Task bidding behavior: At the beginning of the simulation, agents exhibited exploratory behavior, submitting bids for nearly all available tasks regardless of skill match. Specifically, in round 1, the system recorded a global task bid rate of 92%, indicating that most agents evaluated potential utility as positive even for tasks with marginal capability alignment. However, as agents received feedback in the form of success scores, reputation updates, and capability weight adjustments, they became more discerning. By round 20, the global bid rate declined to 72.9%, and and further declined to 55% by round 40 and subsequent rounds. This drop reflects a learned strategy: agents increasingly avoided tasks for which they were underqualified or overloaded, thereby reducing unnecessary task contention and failure. The downward trend in bid rate (see Fig. 5(a)) supports the hypothesis that agents adapted their bidding decisions based on accumulated experience and utility optimization.

Utility improvement: Our utility formulation, which penalizes capability mismatch and overload, encouraged agents to specialize and improve task selection. The average per-agent utility rose from 3.34 in round 1 to 7.0 by round 46, with standard deviation narrowing from 2.41 to 1.92, indicating that utility gains became more consistent across agents (see Fig. 5(b)). These gains were not due to increased task rewards (which remained fixed per dataset) but rather improved alignment between assigned tasks and the agents' skill profiles. High-utility agents typically achieved both high capability match scores and stable reputations, resulting in more frequent task selection and reward accrual.

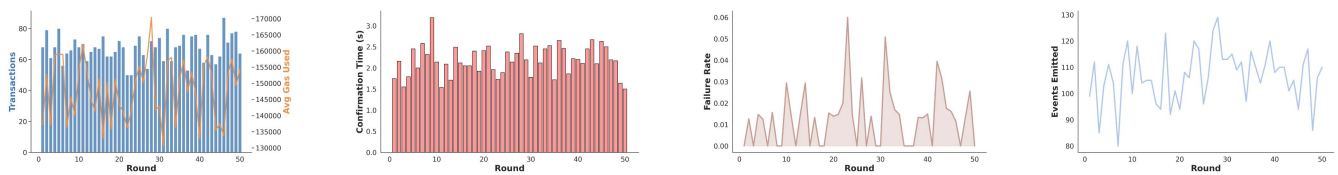
Emergence of specialization: A core effect of our feedback-driven capability adjustment rule is the emergence of domain specialization. We measured each agent's dominant capability tag based on their final capability weight vector and aggregated expert counts across all 10 tags. By round 50, a distinct clustering emerged: 4 agents (20%) specialized in Tag_2, and another 3

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.



(a) Task Bid Rate over Rounds (b) Mean Utility over Rounds (c) Polar Distribution of Capability Tag

Fig. 5. Analysis of agent bidding behavior and capability specialization. (a) Task bid rate decreases over time, suggesting increased agent selectivity; (b) Mean utility steadily increases, indicating improved task-agent alignment; (c) Polar plot illustrates the distribution of dominant capability tags among agents.



(a) Tx Throughput & Gas Usage (b) Confirmation Time (c) Fault Tolerance (d) Event Emission

Fig. 6. Overview of blockchain-layer metrics during LLM-MAS operation: (a) number of events emitted per round, (b) transaction throughput and gas usage trends, (c) confirmation latency over time, and (d) failure rate under operational uncertainty.

agents (15%) became dominant experts in Tag_7. These two domains together accounted for 35% of all specialists, confirming a self-organized concentration of expertise driven by positive reinforcement. Other tags also saw expert accumulation (e.g., Tag_4, Tag_9). The final distribution of tag dominance is visualized in Fig. 5(c).

D. On-Chain Performance and Smart Contract Analysis

To assess the performance of our blockchain-based execution framework, we collected and analyzed key operational metrics over 50 simulation rounds. These include transaction throughput, gas usage, confirmation latency, failure rate, and event emission patterns.

Transaction throughput and gas consumption (cf. Fig. 6(a)): On average, the system processed approximately 67.32 transactions per round, covering agent registration, task submission, assignment, behavioral logging, and reputation or capability updates. The average gas consumption per transaction was 147,865 units, with values ranging from 130,599 to 170,391 units (Table VI). To make these figures more tangible for readers outside the blockchain community, we additionally report an approximate real-world cost estimate under a representative low-congestion setting, assuming a gas price of 5 Gwei and an ETH price of $2,000 (illustrative values). Under this assumption, incentive-critical operations such as task assignment and reputation updates incur execution costs on the order of $0.5–$1 per call. While these costs are substantially lower than peak mainnet conditions, they nevertheless represent a non-negligible economic overhead, which motivates our design choice to restrict blockchain usage to incentive-relevant state transitions rather than fine-grained agent coordination. We note that actual deployment costs depend on network conditions and market prices, and the reported values should be interpreted as order-of-magnitude estimates rather than fixed costs.

TABLE VI
GAS USE AND ESTIMATED EXECUTION COST OF CORE CONTRACT FUNCTIONS

Contract Function	Avg. Gas	Call Freq.	Cost(USD)	Complexity
registerAgent	28,860	20	$0.29	Low
submitTask	40,390	250	$0.40	Medium
assignTask	86,575	250	$0.87	High
logAction	26,772	820	$0.27	Low
updateReputation	61,284	245	$0.61	Medium
updateCapability	58,920	240	$0.59	Medium
TaskAssigned	10,345	250	$0.10	Low
TaskCompleted	13,248	245	$0.13	Low

Confirmation time and responsiveness (cf. Fig. 6(b)): The average confirmation time across rounds was 2.18 seconds, with the fastest confirmations completing in 1.5 seconds and the slowest at 3.19 seconds. These low-latency results suggest that the system remains responsive under moderate agent-task traffic and is suitable for real-time applications where timely task allocation is essential.

Fault tolerance and event emission (cf. Fig. 6(c) & (d)): The average number of failed transactions per round was low at 0.86, with no rounds exceeding 3 failures. This indicates a high reliability of the execution pipeline and proper validation of agent inputs before on-chain invocation. The average number

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

of emitted events per round was 106.82, which includes task broadcasts, bids, assignments, and logging actions. These logs serve as verifiable traces and enable external auditors to reconstruct full histories from immutable ledger records.

Discussion: scalability and economic feasibility: While blockchain-based enforcement provides strong guarantees of transparency, auditability, and trust minimization, it also introduces inherent limitations in terms of transaction cost and throughput. Our experimental results indicate that incentive-relevant operations such as task assignment, behavioral logging, and reputation or capability updates incur non-trivial gas consumption, even with optimized contract logic. This observation highlights a fundamental tension: deploying all coordination logic fully on-chain would be economically infeasible for large-scale agent collaboration under current public blockchain pricing models. Accordingly, the reported gas costs should not be interpreted as evidence that blockchain offers a cost-free solution to decentralized coordination, but rather as a quantification of the overhead required to maintain verifiable incentive enforcement.

To address this limitation, our framework treats the blockchain as a selective enforcement layer rather than a high-throughput execution substrate. Only state transitions that affect long-term incentives and accountability (e.g., task assignment finalization, performance evidence anchoring, and reputation or capability updates) are committed on-chain, while computation-intensive operations such as LLM inference, fine-grained communication, and intermediate reasoning are executed off-chain. This hybrid design significantly reduces on-chain load while preserving the transparency necessary for incentive alignment. Nevertheless, scalability remains an inherent constraint: as the number of agents or tasks grows, cumulative on-chain costs may still become prohibitive if updates are performed too frequently. Potential mitigation strategies include batching incentive updates, periodic settlement of reputation changes, or deployment on lower-cost execution environments such as layer-2 rollups or permissioned blockchains, which we leave as important directions for future work.

VII. Related Work

A. LLM-Based Multi-Agent Systems

General-purpose coordination frameworks: Recent years have witnessed a surge of interest in LLM-based multi-agent systems (LLM-MAS), driven by the observation that groups of specialized agents can outperform monolithic models on complex tasks. Early frameworks such as AutoGPT, BabyAGI, and LangGraph explored agent chaining and tool-augmented reasoning. Subsequent systems formalized agent roles and workflows. MetaGPT [7], ChatDev [41], and AgentVerse [9] adopt role-based or organization-inspired structures to coordinate agents in software engineering tasks. AgentLite [42] and DyLAN [43] emphasize modularity and dynamic agent selection, enabling runtime adaptation.

Other works investigate alternative coordination paradigms. SoT [44] accelerates reasoning through parallel skeleton construction. DMAS [45] adopts a decentralized peer-to-peer topology, while ACORM [46] relies on a centralized controller. CAMEL [47] demonstrates how structured prompts alone can induce cooperative behavior. CORY [48] integrates multi-agent reinforcement learning to co-evolve agent policies.

Domain-specific agent collectives: Beyond general frameworks, LLM-MAS have been deployed in domain-specific settings. In software engineering, multi-agent systems automate coding, testing, security analysis, and penetration testing through specialized agents [49], [50], [51], [52]. In robotics and embodied AI, LLM agents are combined with low-level controllers to enable planning, navigation, and manipulation [53], [54], [55], [56], [57]. In scientific discovery, agent collectives assist with hypothesis generation, experiment planning, and structured debate [58], [59], [60].

Limitations: Despite impressive empirical results, most existing LLM-MAS operate in closed or semi-centralized settings. Trust assumptions are implicit, agent behavior is largely guided by prompt engineering or heuristics, and there is little support for verifiable accountability or incentive-compatible coordination in open environments.

B. Incentive and Alignment Mechanisms for LLMs

Internal alignment via reward modeling: A large body of work studies how to align LLM behavior with human preferences through internal learning signals. RLHF remains the dominant paradigm, using learned reward models and reinforcement learning [61], [62]. More recent approaches such as DPO [63], [64] and its variants [65], [66] simplify optimization by embedding preferences directly into the training objective. Other lines of work refine reward granularity through process-level rewards [67], hierarchical rewards [68], or multi-objective alignment schemes [69], [70].

These methods primarily focus on *single-model alignment* during training or fine-tuning, rather than strategic interactions among multiple autonomous agents.

External incentives and game-theoretic modeling: When LLMs interact with other agents or humans, external incentives become critical. Game-theoretic analyses model LLMs as strategic actors in markets, communication games, or adversarial settings [71], [72], [73], [74]. Mechanism design has been applied to LLM-mediated auctions [75], data markets and fine-tuning services [76], [77], and peer-evaluation systems [78]. Self-play and adversarial training further explore how competition shapes behavior [79].

However, these studies typically analyze incentives *in isolation*, without integrating them into a full multi-agent coordination loop with persistent reputation and task allocation.

C. Lessons Learned

Based on the above literature, we distill several key lessons:

- *Coordination alone is insufficient:* Existing LLM-MAS focus on communication, role assignment, or workflow design, but lack explicit incentive mechanisms to regulate strategic behavior in open settings.
- *Internal alignment does not address multi-agent incentives:* Reward-model-based alignment techniques shape

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

individual model behavior, but do not govern interactions among autonomous agents with competing objectives.

- *Game-theoretic incentives lack system integration:* Prior incentive and mechanism design studies analyze specific interaction scenarios, yet rarely connect incentives to end-to-end agent coordination, task allocation, and long-term trust.
- *Trust and accountability remain underexplored:* Few LLM-MAS provide verifiable guarantees on agent identity, behavior logging, and incentive enforcement in decentralized environments.

VIII. Conclusion and Future Work

This work formulates decentralized LLM-based multi-agent coordination as a *behavior-shaping incentive design* problem. Our primary contribution is an incentive mechanism that aligns agent behavior over repeated interactions through utility-based task allocation, performance-driven reputation updates, and adaptive capability modeling. Simulation results show that this mechanism promotes cooperative behavior, discourages strategic misreporting and overload, and leads to emergent specialization among agents. The blockchain layer functions as a supporting enforcement substrate that operationalizes these incentive rules in a transparent and verifiable manner.

Several directions remain for future work. First, a more comprehensive economic analysis under dynamic pricing and strategic participation is needed. Second, extending the framework to adversarial or partially malicious agents would enable deeper game-theoretic investigation. Finally, large-scale deployment will require tighter off-chain integration and cost-aware enforcement mechanisms.

Appendix

A. Notation Table

To facilitate a clear understanding of the system's mechanisms and mathematical formulations, Table VII summarizes all the key symbols used throughout the paper. The notation covers core concepts such as agent capabilities, task properties, reputation dynamics, utility modeling, assignment probabilities, etc. We also include relevant smart contract function names and emitted events in the table.

TABLE VII
Notation Table for Multi-Agent Framework

Symbol	Description
A_i	The i-th agent in the system
T_j	The j-th task to be assigned
$\mathcal{A}$	Set of all agents in the system
$\mathcal{T}$	Global taxonomy of capability tags
$\mathbf{r}_j$	Binary vector for required capabilities for task T_j
$\boldsymbol{w}_i^t$	Continuous capability weight vector of agent A_i at time t
$\hat{\mathbf{w}}_i^t$	Binarized capability vector derived from w_i^t
$w_{i,k}^t$	Weight for capability tag k of agent A_i at time t
ρ_i^t	Reputation score of agent A_i at time t
ℓ_i^t	Task workload for agent A_i at time t
$\pi_i^t(T_j)$	Assignment probability of task T_j to agent A_i at time t
R_j	Reward associated with task T_j
$U_i^t(T_j)$	Expected utility of agent A_i for task T_j at time t
$c_i^t(T_j)$	Cost incurred by agent A_i when executing task T_j
S_i^t	Task performance score received by agent A_i at time t
q_i^t	Quality component of task score S_i^t
d_i^t	Delay ratio component of task score S_i^t
α	Smoothing coefficient for reputation update
μ	Smoothing coefficient for capability weight update
β, γ	Cost parameters for workload and skill mismatch
θ	Threshold for capability binarization
$\mathsf{CapMatch}(T_j, A_i)$	Capability match score between task T_j and agent A_i
$\mathsf{Score}(T_j, A_i)$	Final agent selection score for task T_j and agent A_i
Smart Contract Functions and Events	
registerAgent	Function to register an agent on-chain with metadata
submitTask	Function to publish a new task to the blockchain
assignTask	Executes task assignment policy and emits allocation event
logAction	Logs agent behavior and intermediate states
updateReputation	Updates agent's reputation based on task performance
updateCapability	Updates agent's capability weights after task completion
TaskAssigned	Event emitted when a task is assigned to an agent
TaskCompleted	Event emitted when task execution is reported

B. Hyperparameter Configuration

To ensure reproducibility of our experiments, we list the key hyperparameters used in the incentive mechanism and task allocation process. Unless otherwise stated, all parameters are fixed across all simulation rounds.

Task allocation scoring weights: In the task allocation scoring function (8), we set the weights to $\lambda_1 = 0.5, \lambda_2 = 0.3, \lambda_3 = 0.2$, placing the highest emphasis on capability matching, followed by reputation, and then workload. This choice reflects the intuition that task–skill alignment is the primary determinant of execution quality, while reputation captures longer-term reliability and workload discourages agent overcommitment.

Utility cost coefficients: In the cost function (2), we use $\beta = 0.1$ for workload cost and $\gamma = 0.4$ for capability mismatch penalty. This configuration ensures that agents are strongly discouraged from accepting tasks for which they lack required skills, while moderate workload accumulation is tolerated to allow parallelism.

Reputation and capability smoothing parameters: For the exponential update rules of reputation (4) and capability weights (7), we set the smoothing coefficients to $\lambda = 0.7$ and $\mu = 0.6$, respectively. These values balance stability and adaptability: reputation evolves more conservatively to reflect long-term trust, whereas capability weights adapt more quickly to recent task-specific performance.

References

[1] T. Yang et al., "AutoHMA-LLM: Efficient task coordination and execution in heterogeneous multi-agent systems using hybrid large language models," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 2, pp. 987–998, Apr. 2025.

[2] L. Zhou et al., "Semantic information extraction and multi-agent communication optimization based on generative pre-trained transformer," *IEEE Trans. Cogn. Commun. Netw.*, vol. 11, no. 2, pp. 725–737, Apr. 2025.

[3] Z. Yao, Z. Tang, W. Yang, and W. Jia, "Enhancing LLM QoS through cloud-edge collaboration: A diffusion-based multi-agent reinforcement learning approach," *IEEE Trans. Serv. Comput.*, vol. 18, no. 3, pp. 1412–1427, May/Jun. 2025.

[4] C. Sun, S. Huang, and D. Pompili, "LLM-based multi-agent decision-making: Challenges and future directions," *IEEE Robot. Automat. Lett.*, vol. 10, no. 6, pp. 5681–5688, Jun. 2025.

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

[5] M. Qi, D. He, Q. Wang, and L. Zhang, "Viper strike: Defeating visual reasoning captchas via structured vision-language inference," 2026, *arXiv:2601.06461*.

[6] Z. Wei et al., "Advanced smart contract vulnerability detection via LLM-powered multi-agent systems," *IEEE Trans. Softw. Eng.*, vol. 51, no. 10, pp. 2830–2846, Oct. 2025.

[7] S. Hong et al., "MetaGPT: Meta programming for a multi-agent collaborative framework.," in *Proc. Int. Conf. Learn. Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=VtmBAGCN7o

[8] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation framework," 2023, *arXiv:2308.08155*.

[9] W. Chen et al., "AgentVerse: Facilitating multi-agent collaboration and exploring emergent behaviors in agents," in *Proc. 12th Int. Conf. Learn. Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=EHg5GDnyq1

[10] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, "AgentCoder: Multi-agent-based code generation with iterative testing and optimisation," 2023, *arXiv:2312.13010*.

[11] T. Guo et al., "Large language model based multi-agents: A survey of progress and challenges," in *Proc. 33rd Int. Joint Conf. Artif. Intell.*, 2024, pp. 8048–8057.

[12] K. Cinkusz and J. A. Chudziak, "Towards LLM-augmented multiagent systems for agile software engineering," in *Proc. 39th IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2024, pp. 2476–2477.

[13] W. Epperson et al., "Interactive debugging and steering of multi-agent ai systems," in *Proc. 2025 CHI Conf. Hum. Factors Comput. Syst.*, 2025, pp. 1–15.

[14] L. Zhang, Y. Zhai, T. Jia, X. Huang, C. Duan, and Y. Li, "AgentFM: Role-aware failure management for distributed databases with LLM-driven multi-agents," in *Proc. 33rd ACM Int. Conf. Found. Softw. Eng.*, 2025, pp. 525–529.

[15] H. Liang, Z. Chang, and C. K. Ahn, "Hybrid event-triggered intermittent control for nonlinear multi-agent systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 4, pp. 1975–1984, Jul./Aug. 2023.

[16] Q. Pan, J. Wu, J. Li, W. Yang, and M. Guizani, "Blockchain and multi-agent learning empowered incentive irs resource scheduling for intelligent reconfigurable networks," *IEEE/ACM Trans. Netw.*, vol. 32, no. 2, pp. 943–958, Apr. 2024.

[17] L. Yuan et al., "Multi-agent incentive communication via decentralized teammate modeling," in *Proc. AAAI Conf. Artif. Intell.*, 2022, pp. 9466–9474.

[18] A. Jin, Y. Ye, B. Lee, and Y. Qiao, "DecoAgent: Large language model empowered decentralized autonomous collaboration agents based on smart contracts," *IEEE Access*, vol. 12, pp. 155234–155245, 2024.

[19] B. Chen, G. Li, X. Lin, Z. Wang, and J. Li, "BlockAgents: Towards byzantine-robust LLM-based multi-agent coordination via blockchain," in *Proc. ACM Turing Award Celebration Conf. China*, 2024, pp. 187–192.

[20] Z.-K. Chong, H. Ohsaki, and B. Ng, "LLM-Net: Democratizing LLMs-as-a-Service through blockchain-based expert networks," in *Proc. 14th Int. Conf. Software Comput. Appl.*, 2025, pp. 313–320.

[21] M. Qi et al., "SOK: Bitcoin layer two (l2)," *ACM Comput. Surv.*, vol. 58, pp. 1–37, 2024.

[22] D. Xiao, C. Zhang, H. Deng, J. Liang, L. Wang, and L. Zhu, "Parallelizing universal atomic swaps for multi-chain cryptocurrency exchanges," in *Proc. 34th USENIX Secur. Symp.*, 2025, pp. 4073–4092.

[23] Z. Li, S. Gao, Z. Peng, S. Guo, Y. Yang, and B. Xiao, "B-DNS: A secure and efficient DNS based on the blockchain technology," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1674–1686, Apr./Jun. 2021.

[24] P. Kumar, R. Kumar, A. Kumar, A. A. Franklin, S. Garg, and S. Singh, "Blockchain and deep learning for secure communication in digital twin empowered industrial IoT network," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2802–2813, Sep./Oct. 2023.

[25] N. Li et al., "Blockchain cross-chain bridge security: Challenges, solutions, and future outlook," *Distrib. Ledger Technol., Res. Pract.*, vol. 4, no. 1, pp. 1–34, 2025.

[26] C. Jiang, X. Liu, S. Wang, J. Liu, and Y. Zhang, "EFEVD: Enhanced feature extraction for smart contract vulnerability detection," in *Proc. 33rd Int. Joint Conf. Artif. Intell.*, 2024, pp. 4246–4254.

[27] R. Wang, Y. Zhang, and L. Peng, "Anomaly detection service for blockchain transactions using minimal substitution-based label propagation," *IEEE Trans. Serv. Comput.*, vol. 17, no. 5, pp. 2054–2066, Sep./Oct. 2024.

[28] M. Qi, Z. Xu, Z. Wang, S. Chen, and Y. Xiang, "Databox-based delivery service via blockchain," in *Proc. IEEE Int. Conf. Web Serv.*, 2022, pp. 79–84.

[29] Z. Wang, Y. Yu, W. Zheng, W. Ma, and M. Zhang, "MACREC: A multi-agent collaboration framework for recommendation," in *Proc. 47th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2024, pp. 2760–2764.

[30] B. Jiang, Y. Xie, X. Wang, W. J. Su, C. J. Taylor, and T. Mallick, "Multi-modal and multi-agent systems meet rationality: A survey," in *Proc. Int. Conf. Mach. Learn. Workshop LLMs Cogn.*, 2024. [Online]. Available: https://openreview.net/forum?id=9Rtm2gAVjo

[31] D. Chen, X. Liu, and W. Yu, "Finite-time fuzzy adaptive consensus for heterogeneous nonlinear multi-agent systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 3057–3066, Oct./Dec. 2020.

[32] P. P. Santos et al., "Centralized training with hybrid execution in multi-agent reinforcement learning via predictive observation imputation," *Artif. Intell.*, vol. 348, 2025, Art. no. 104404.

[33] Z. Pan et al., "Robust watermarking for federated diffusion models with unlearning-enhanced redundancy," *IEEE Trans. Dependable Secure Comput.*, no. 1, pp. 1–15, 2025.

[34] H. Zhao, Z. Pan, Y. Wang, Z. Ying, L. Xu, and Y.-A. Tan, "Personalized label inference attack in federated transfer learning via contrastive meta learning," in *Proc. AAAI Conf. Artif. Intell.*, 2025, pp. 22777–22785.

[35] J. Lim and P. Tsiotras, "CBS-budget (CBSB): A complete and bounded suboptimal search for multi-agent path finding," *Artif. Intell.*, vol. 346, 2025, Art. no. 104349.

[36] X. Ren, H. Liang, Y. Wang, C. Zhang, Z. Xiong, and L. Zhu, "BESA: Boosting encoder stealing attack with perturbation recovery," *IEEE Trans. Inf. Forensics Secur.*, vol. 20, pp. 10007–10018, 2025.

[37] Z. Pan et al., "Feature-based machine unlearning for vertical federated learning in IoT networks," *IEEE Trans. Mobile Comput.*, vol. 24, no. 6, pp. 5031–5044, Jun. 2025.

[38] Z. Xu et al., "The trust paradox in LLM-based multi-agent systems: When collaboration becomes a security vulnerability," 2025, *arXiv:2510.18563*.

[39] F. Guan, T. Zhu, W. Chang, W. Ren, and W. Zhou, "Large language models merging for enhancing the link stealing attack on graph neural networks," *IEEE Trans. Dependable Secure Comput.*, vol. 22, no. 6, pp. 6809–6825, Nov./Dec. 2025.

[40] M. Shridhar et al., "Alfred: A benchmark for interpreting grounded instructions for everyday tasks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 10740–10 749.

[41] C. Qian et al., "Communicative agents for software development," in *Proc. 62nd Annu. Meeting Assoc. Comput. Linguistics*, 2024, pp. 15174–15186.

[42] Z. Liu, W. Yao, J. Zhang, L. Yang, Z. Liu, and J. Tan, "Agentlite: A lightweight library for building and advancing task-oriented LLM agent system," 2024, *arXiv:2402.15538*.

[43] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, "Dynamic LLM-agent network: An LLM-agent collaboration framework with agent team optimization," 2023, *arXiv:2310.02170*.

[44] X. Ning, Z. Lin, Z. Zhou, Z. Wang, H. Yang, and Y. Wang, "Skeleton-of-thought: Prompting LLMs for efficient parallel generation," in *Proc. 12th Int. Conf. Learn. Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=mqVgBbNCm9

[45] Y. Chen, J. Arkin, Y. Zhang, N. Roy, and C. Fan, "Scalable multi-robot collaboration with large language models: Centralized or decentralized systems?," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2024, pp. 4311–4317.

[46] Z. Hu, Z. Zhang, H.-Y. Li, C. Chen, H. Ding, and Z. Wang, "Attention-guided contrastive role representations for multi-agent reinforcement learning," in *Proc. 12th Int. Conf. Learn. Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=LWmuPfEYhH

[47] G. Li, H. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "Camel: Communicative agents for" mind" exploration of large scale language model society," *Adv. Neural Inf. Process. Syst.*, vol. 36, pp. 51 991–52 008, 2023.

[48] H. Ma et al., "Coevolving with the other you: Fine-tuning LLM with sequential cooperative multi-agent reinforcement learning," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 15497–15525.

[49] W. Tao, Y. Zhou, Y. Wang, W. Zhang, H. Zhang, and Y. Cheng, "Magis: LLM-based multi-agent framework for github issue resolution," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 51963–51993.

[50] S. Kang, J. Yoon, and S. Yoo, "Large language models are few-shot testers: Exploring LLM-based general bug reproduction," in *Proc. IEEE/ACM 45th Int. Conf. Softw. Eng.*, 2023, pp. 2312–2323.

[51] G. Deng et al., "{PentestGPT}: Evaluating and harnessing large language models for automated penetration testing," in *Proc. 33rd USENIX Secur. Symp.*, 2024, pp. 847–864.

[52] Y. Dong, X. Jiang, Z. Jin, and G. Li, "Self-collaboration code generation via chatgpt," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, pp. 1–38, 2024.

[53] Z. Mandi, S. Jain, and S. Song, "Roco: Dialectic multi-robot collaboration with large language models," in *Proc. 2024 IEEE Int. Conf. Robot. Automat.*, Yokohama, Japan, 2024, pp. 286–299.

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.

[54] H. Zhang et al., "Building cooperative embodied agents modularly with large language models," in *Proc. Int. Conf. Neural Inf. Process. Syst. Found. Models Decis. Mak. Workshop*, New Orleans, LA, USA, 2024. [Online]. Available: https://openreview.net/forum?id=EnXJfQqy0K

[55] S. S. Kannan, V. L. Venkatesh, and B.-C. Min, "Smart-LLM: Smart multi-agent robot task planning using large language models," in *Proc. 2024 IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2024, pp. 12140–12147.

[56] G. Zhu, R. Zhou, W. Ji, and S. Zhao, "LamaRL: LLM-aided multi-agent reinforcement learning for cooperative policy generation," *IEEE Robot. Automat. Lett.*, vol. 10, no. 7, pp. 7476–7483, Jul. 2025.

[57] T. Yao, Y. Xu, H. Wang, X. Qiu, K. Althoefer, and P. Qi, "Multi-agent fuzzy reinforcement learning with LLM for cooperative navigation of endovascular robotics," *IEEE Trans. Fuzzy Syst.*, pp. 1–11, 2025.

[58] A. M. Bran, S. Cox, O. Schilter, C. Baldassari, A. D. White, and P. Schwaller, "Chemcrow: Augmenting large-language models with chemistry tools," *Nature Mach. Intell.*, vol. 6, no. 5, pp. 525–535, 2024.

[59] A. Ghafarollahi and M. J. Buehler, "Protagents: Protein discovery via large language model multi-agent collaborations combining physics and machine learning," *Digit. Discovery*, vol. 3, no. 7, pp. 1389–1409, 2024.

[60] T. Liang et al., "Encouraging divergent thinking in large language models through multi-agent debate," in *Proc. 2024 Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 17889–17904.

[61] Y. Cao et al., "Survey on large language model-enhanced reinforcement learning: Concept, taxonomy, and methods," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 36, no. 6, pp. 9737–9757, Jun. 2025.

[62] F. Peiyuan et al., "Agile: A novel reinforcement learning framework of LLM agents," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 5244–5284.

[63] R. Rafailov, A. Sharma, E. Mitchell, S. Ermon, C. D. Manning, and C. Finn, "Direct preference optimization: Your language model is secretly a reward model," in *Proc. 37th Int. Conf. Neural Inf. Process. Syst.*, 2023, pp. 53728–53741.

[64] S. Xu et al., "Is DPO superior to PPO for LLM alignment? A comprehensive study," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 54983–54998.

[65] J. Wu et al., "δ-DPO: Direct preference optimization with dynamic δ," in *Proc. 38th Int. Conf. Neural Inf. Process. Syst.*, 2023, pp. 129944–129966.

[66] W. Zeng, Y. Lan, J. He, and G. Neubig, "Trust region direct preference optimization," *Adv. Neural Inf. Process. Syst.*, vol. 37, pp. 27 463–27 489, 2024.

[67] H. Lightman et al., "Let's verify step by step," in *Proc. 12th Int. Conf. Learn. Representations*, 2023. [Online]. Available: https://openreview.net/forum?id=v8L0pN6EOi

[68] Y. Li et al., "Hierarchical reward models for long-form text generation," *Findings Assoc. Comput. Linguistics*, 2024, pp. 7817–7831.

[69] H. Wang, W. Xiong, T. Xie, H. Zhao, and T. Zhang, "Interpretable preferences via multi-objective reward modeling and mixture-of-experts," in *Proc. Conf. Assoc. Comput. Linguistics - Empir. Methods Natural Lang. Process.*, Miami, FL, USA, 2024, pp. 10582–10592.

[70] M. Qi, Z. Wang, S. Chen, and Y. Xiang, "A hybrid incentive mechanism for decentralized federated learning," Distributed Ledger Technologies: *Res. Pract.*, vol. 1, no. 1, pp. 1–15, 2022.

[71] F. Yao, C. Li, H. Li, Y. Ghale, and Z.-L. Zhang, "Human vs. generative AI in content creation competition: Symbiosis or conflict?," in *Proc. 41st Int. Conf. Mach. Learn.*, 2024, pp. 56885–56913.

[72] L. Zhang, T. Zhu, P. Xiong, W. Zhou, and P. S. Yu, "A robust game-theoretical federated learning framework with joint differential privacy," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 4, pp. 3333–3346, Apr. 2023.

[73] C. Fan, J. Chen, Y. Jin, and H. He, "Can large language models serve as rational players in game theory? a systematic analysis," in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 17960–17967.

[74] S. Mao et al., "Alympics: LLM agents meet game theory," in *Proc. 31st Int. Conf. Comput. Linguistics*, 2025, pp. 2845–2866.

[75] P. Duetting et al., "Mechanism design for large language models," in *Proc. ACM Web Conf.*, 2023, pp. 144–155.

[76] B. Laufer, J. Kleinberg, and H. Heidari, "Fine-tuning games: Bargaining and adaptation for general-purpose models," in *Proc. ACM Web Conf.*, 2023, pp. 66–76.

[77] Y. Liu, Y. Jia, J. Jia, D. Song, and N. Z. Gong, "DataSentinel: A game-theoretic detection of prompt injection attacks," in *Proc. IEEE Symp. Secur. Privacy*, 2025, pp. 2190–2208.

[78] J. Duan et al., "GTBench: Uncovering the strategic reasoning capabilities of LLMs via game-theoretic evaluations," in *Proc. Adv. Neural Inf. Process. Syst.*, 2024, pp. 28219–28253.

[79] E. Akata, L. Schulz, J. Coda-Forno, S. J. Oh, M. Bethge, and E. Schulz, "Playing repeated games with large language models," *Nature Hum. Behav.*, vol. 9, pp. 1380–1390, 2025.

Minfeng Qi (Member, IEEE) received the M.Eng. degree in information systems from Monash University, Clayton, VIC, Australia, in 2019, and the Ph.D. degree in computer science from the Swinburne University of Technology, Hawthorn, VIC, Australia, in 2023. He is currently an Assistant Professor with the City University of Macau, Macau, China. His research interests include blockchain security and AI security.

Tianqing Zhu (Member, IEEE) received the B.Eng. degree in chemistry and the M.Eng. degree in automation from Wuhan University, Wuhan, China, in 2000 and 2004, respectively, and the Ph.D. degree in computer science from Deakin University, Docklands, VIC, Australia, in 2014. She is currently a Professor with the Faculty of Data Science, City University of Macau, Macau, China. Her research interests include privacy-preserving, cybersecurity, and privacy in artificial intelligence.

Lefeng Zhang received the B.Eng. and M.Eng. degrees from the Zhongnan University of Economics and Law, Wuhan, China, in 2016 and 2019, respectively, and the Ph.D. degree from the University of Technology Sydney, Ultimo, NSW, Australia, in 2024. He is currently an Assistant Professor with the City University of Macau, Macau, China. His research interests include game theory and privacy-preserving.

Ningran Li (Graduate Student Member, IEEE) received the B.Sc. degree in information systems, the M.Sc. degree in information technology, and the Ph.D. degree in computer science from the Swinburne University of Technology, Melbourne, Australia, in 2024. She is currently an Assistant Professor with the University of Adelaide, Adelaide, SA, Australia. Her research interests mainly include cybersecurity, blockchain security, and cross-chain technologies.

Yu-an Tan (Senior Member, IEEE) received the B.Eng. degree in computer software in 1991 and the Ph.D. degree in computer science in 2003. He has Teaching and Research Experience of more than 26 years, and has been a Professor with Beijing Institute of Technology, Beijing, China, since 2010. He contributes for peer-reviewed more than 100 journal articles and conference papers, including five papers of the top 1% of Essential Science Indicators. His research interests include artificial intelligence security, cybersecurity, and storage sub-systems. He is a Senior Member of China Computer Federation. He was the recipient of more than 20 research funds from the National Natural Science Foundation of China and the National Key Research and Development Program of China.

Wanlei Zhou (Life Fellow, IEEE) received the B.Eng. and M.Eng. degrees in computer science and engineering from Harbin Institute of Technology, Harbin, China, in 1982 and 1984, respectively, the Ph.D. degree in computer science and engineering from The Australian National University, Canberra, ACT, Australia, in 1991, and the D.Sc. degree (Higher Doctoral degree) from Deakin University, VIC, Australia, in 2002. He is currently the Vice Rector of the City University of Macau, Macau, China. He has authored or coauthored more than 400 papers in refereed international journals and refereed international conferences proceedings, including many articles in IEEE transactions and journals. His research interests include security, privacy-preserving, and distributed systems.

Authorized licensed use limited to: Indian Institute of Information Technology Kota. Downloaded on March 06,2026 at 18:09:24 UTC from IEEE Xplore. Restrictions apply.